

程式設計(二)期末時做 報告

2026/5/22

S11459042 金浩樑

Github 連結:

[https://github.com/mm0989842208-
sudo/0622_final_Practical.git](https://github.com/mm0989842208-sudo/0622_final_Practical.git)

1. 自主學習主題與目標

主題： STL 與基礎資料結構入門

目標： 本次自主學習接續 Chapter 16 Templates 的觀念，進一步認識 C++ 標準模板函式庫 (STL)。透過熟悉常見的 STL 容器 (如 vector、map、stack、queue) 及其背後的基礎資料結構，最終能根據問題需求選擇合適的容器，並完成「學生成績管理系統」的整合應用小專題，為未來的大二資料結構課程建立先備基礎。

2. Template 與 STL 的關係

Template (模板) 是 C++ 中實現「泛型編程 (Generic Programming)」的核心機制，它允許我們撰寫與「資料型別」無關的通用程式碼。

STL (Standard Template Library) 的本質上就是 Template 的極致應用。C++ 標準委員會利用 class template 和 function template 的技術，將常用的資料結構與演算法封裝起來。因此，我們才能使用同一個 `std::vector` 容器，隨意裝入 `int`、`string`，甚至是自訂的結構 (ex: `Student`)，完美體現了 Template 讓程式碼能套用於不同型別的強大優勢。

3. 常見 STL 容器用途整理

常見的 STL 容器特性：

- **vector (動態陣列)**：在記憶體中連續配置，支援快速的隨機存取與尾部新增/刪除。適合儲存需要頻繁走訪且大小會變動的資料。
- **stack (堆疊)**：具備後進先出特性，只能從頂端存取資料，適合用於檢查括號配對或某些演算法的回溯。
- **queue (佇列)**：具備先進先出 (FIFO) 特性，從尾部加入、從頭部取出，適合用來模擬排隊系統。
- **map (映射表)**：儲存鍵值對 (Key-Value) 的關聯式容器，適合用來計算特定元素出現的次數（如單字統計），或以特定 ID（如學號）進行快速查詢。

4. 第 2 週小專題設計說明

本專題設計了一個命令列介面 (CLI) 的「學生成績管理系統」，提供新增、列出、排序、查詢與統計等 5 大功能。在資料結構的選擇上，考量到學生成績管理需要頻繁地將所有資料印出與排序，因此選用 `std::vector<Student>` 作為主要的資料容器。為了確保資料正確性，在新增資料時實作了防呆機制，若偵測到學號重複，將提示錯誤並拒絕新增覆蓋。且在重要段落有做中文註解以便他人理解我的程式碼。

5. 主要程式架構與重要程式片段

解我的程式碼系統採用 do-while 迴圈搭配 switch-case 來驅動主選單，並在每次選擇後執行對應的功能。以下為重要的程式實作片段：

- Lambda 表達式進行自訂排序：利用 <algorithm> 函式庫中的 std::sort，並直接在第三個參數使用 Lambda 定義排序規則，將全班成績由高至低排列。

```
sort(students.begin(), students.end(), [](const Student& a, const Student& b) {  
    return a.score > b.score;  
});
```

- Template Function 應用 (固定挑戰題)：定義了泛型函式 getMax 與 getMin，並實際呼叫運用在「統計成績」功能中計算全班最高與最低分。其中也用了比 if else 迴圈更節省的三元運算子去用新方法優化我的程式。

```
template <class T>  
T getMax(T a, T b) {  
    return (a > b) ? a : b;  
}  
  
template <class T>  
T getMin(T a, T b) {  
    return (a < b) ? a : b;  
}
```

6. 執行畫面或輸出結果

防呆機制的觸發：

```
=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Pleace choice the function: h
Error.Pleace enter the number!

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Pleace choice the function: 1
Enter student ID: S004
Enter name: studentD
Enter score: 60
Successfully added student information!

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Pleace choice the function: 1
Enter student ID: S004
Error! ID: S004 has existed.
```

學生列表與排序功能(已新增學生 A~D)：

```
=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Pleace choice the function: 2

--- Student list ---
ID          Name          Score
-----
S001        studentA        90
S002        studentB        45
S003        studentC        80
S004        studentD        60

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Pleace choice the function: 3
The scores have been sorted from highest to lowest!

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Pleace choice the function: 2

--- Student list ---
ID          Name          Score
-----
S001        studentA        90
S003        studentC        80
S004        studentD        60
S002        studentB        45
```

查詢與統計分析功能展示：

```
=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Place choice the function: 4
Enter the student ID you want to search: S002

find student data:
ID: S002, Name: student_B, Score: 45

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Place choice the function: 4
Enter the student ID you want to search: D888
can't find the ID: D888 in student data.

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Place choice the function: 5

--- 成績統計 ---
全班平均: 68.75
最高分: 90
最低分: 45
及格人數: 3
不及格人數: 1
```

7. GitHub repo link 與檔案結構說明

GitHub Repository 連結：https://github.com/mm0989842208-sudo/0622_final_Practical.git

檔案結構說明：

main.cpp: 學生成績管理系統的主要原始碼，包含所有選單與邏輯實作。

README.md: 專案說明文件，記錄在 g++ -std=c++17 環境下編譯與執行的指令。

report.md: 本份自主學習報告。

8. 遇到的問題與解決方式

選單輸入型別錯誤導致系統崩潰：

問題： 當程式要求輸入選單數字 (int)，若不小心輸入英文字母，cin 會進入錯誤狀態且自動將變數設為 0，導致系統印出錯誤訊息後觸發迴圈結束條件，直接強制退出。

解決方式： 實作了輸入緩衝區的防呆機制。使用 `if (!(cin >> choice))` 捕捉錯誤，接著透過 `cin.clear()` 清除錯誤旗標，並用 `cin.ignore(10000, '\n')` 清空輸入暫存區的垃圾字元，最後強制將 `choice` 設為 -1 以避免意外退出迴圈。

Git 指令拼寫錯誤與強制推送：

問題： 在準備上傳 GitHub 時，將綁定遠端的指令誤打成 `git temote` 導致連結失敗；隨後因為遠端 Repo 包含本地沒有的初始檔案，導致 `git push` 被拒絕 (Updates were rejected)。

解決方式： 仔細閱讀終端機提示，將指令修正為正確的 `git remote add origin`。接著，因為確定本地端的專案才是最終版本，所以使用 `git push -f -u origin main` 強制覆蓋遠端內容，順利完成上傳。

9. 學習心得與未來可延伸方向

學習心得： 透過這次專題，我大幅提升了 C++ 實作能力。以前處理陣列都要手動管理大小，現在利用 STL 的 `vector` 不僅讓資料動態擴充變得直覺，配合 `sort` 演算法更是省下了許多底層實作的負擔，讓我能更專注於系統的商業邏輯。同時，在實作 `Template Function` 與除錯 `cin` 緩衝區的過程中，我更深刻理解了 C++ 型別推導與底層輸入機制的嚴謹性。

未來可延伸方向：

資料持久化： 目前程式結束後資料會遺失，未來可引入 `<fstream>` 將學生資料寫入外部檔案 (如 CSV)，達到存檔與讀取的效果。